Módulo 7- Liderança em Testes de Software

Nome: Mariana Caetano Costa
Email: mariana.cae.costa@gmail.com

OBJETIVO DO MÓDULO

O que você vai aprender:

- **Comportamento do líder:** características e atitudes de uma boa liderança.
- **Relacionamento com a equipe:** como o líder interage com seus liderados.
- **Ferramentas para liderança:** recursos que podem auxiliar na gestão da equipe.
- **Provas de conceito (PoC):** como elaborar e utilizar provas de conceito para avaliar soluções e ideias.

Objetivo da aula: desenvolver habilidades para atuar como um líder melhor, tanto para quem já exerce essa função quanto para quem deseja assumir uma posição de liderança.

Conceito importante: uma prova de conceito (Proof of Concept — PoC) é uma implementação ou experimento inicial usado para verificar se uma ideia ou solução é tecnicamente viável antes de investir em seu desenvolvimento completo.

---

2. LIDERANÇA DE TESTES CONTEMPORÂNEA

1. O que é liderança contemporânea?

A liderança contemporânea atua em um contexto ágil, no qual os QAs trabalham junto com desenvolvedores e outros profissionais do time, com autonomia para organizar e executar suas atividades. Nesse modelo, o líder não precisa controlar cada tarefa. Seu papel é desenvolver pessoas, orientar decisões e ajudar os QAs a se tornarem cada vez mais autônomos e preparados para liderar.

2. Por que precisamos de um líder se o QA é autogerenciável?

Ser autogerenciável não significa saber tudo ou não precisar de apoio. O QA pode ter autonomia para tomar decisões e, ainda assim, precisar de mentoria, treinamento e orientação para evoluir profissionalmente.

O líder contribui para que o QA:

- Desenvolva autonomia e capacidade de decisão.
- Aprimore suas habilidades técnicas e comportamentais.
- Identifique riscos e comunique resultados.
- Melhore seu relacionamento e sua influência dentro do time.
- Escolha estratégias, ferramentas e práticas de teste adequadas.
- Supere dificuldades e desenvolva planos de ação.

Ideia central: o líder contemporâneo não faz tudo pelo QA; ele ajuda o QA a desenvolver a capacidade de fazer boas escolhas por conta própria.

3. Liderança tradicional × liderança contemporânea

Liderança tradicional	**Liderança contemporânea**
Baseada em comando e controle.	Baseada em autonomia e colaboração.
O líder distribui e controla tarefas.	O QA participa das decisões e organiza seu trabalho.
O líder define como e quando as atividades serão feitas.	O time escolhe abordagens adequadas aos seus objetivos.
O liderado responde diretamente às orientações do líder.	O QA atua como protagonista no time ágil.
O líder supervisiona a execução.	O líder orienta, ensina e desenvolve pessoas.

4. Responsabilidades do líder contemporâneo de QA

O líder atua principalmente como mentor, orientador e agente de melhoria.

Mentoria e desenvolvimento

- Compartilhar experiências e lições aprendidas.
- Orientar os QAs na resolução de dificuldades.
- Incentivar o desenvolvimento da autonomia e de novas lideranças.

Acompanhamento do time

- Avaliar a qualidade da comunicação e dos relacionamentos.
- Ajudar a identificar riscos e melhorar as estratégias de teste.
- Orientar o uso adequado de técnicas, ferramentas e abordagens.
- Incentivar relatórios claros e decisões baseadas em evidências.

Melhoria organizacional

- Compreender as necessidades da empresa.
- Identificar lacunas nos processos de qualidade.
- Propor melhorias e ajudar a implementá-las.

5. Exemplo prático em QA

Imagine que um QA tenha dificuldade para definir a estratégia de testes de uma nova funcionalidade.

- Líder tradicional: determina quais testes devem ser executados e como devem ser feitos.
- Líder contemporâneo: ajuda o QA a analisar os requisitos, identificar riscos, avaliar alternativas e escolher a estratégia mais adequada.

No segundo caso, o QA participa da decisão e aprende a resolver problemas semelhantes no futuro.

6. O que você precisa levar desta aula

Como futura líder de QA, seu papel não será apenas garantir que os testes sejam executados. Você também precisará desenvolver pessoas capazes de tomar decisões, influenciar o time, identificar riscos e promover a qualidade do software.

O objetivo é formar profissionais autônomos e capazes de exercer liderança, sem depender de instruções constantes.
Conceito-chave: liderança contemporânea é menos sobre controlar tarefas e mais sobre desenvolver pessoas, fortalecer a autonomia e melhorar a qualidade dentro da organização.

---

3. POLITICA DE TESTES

A política de testes é um documento formal que estabelece os objetivos, princípios e diretrizes que orientam a disciplina de testes em uma organização, projeto ou grupo de times.
O instrutor apresenta um modelo baseado na ISO/IEC/IEEE 29119-3, norma relacionada à documentação de testes de software.
O objetivo é garantir que QAs, desenvolvedores, Product Owners (POs), Scrum Masters e demais profissionais compreendam como a qualidade e os testes são tratados naquele contexto.

1. Estrutura da política de testes

1. Histórico de mudanças
Registra as alterações realizadas no documento, garantindo rastreabilidade e controle de versões.

Informações registradas:

- Identificador sequencial da mudança.
- Autor da alteração.
- Descrição do que foi alterado.
- Data da alteração.
- Responsável pela aprovação.

Por ser um documento formal, as mudanças podem precisar de aprovação de um CTO, gerente ou diretor responsável.

2. Escopo

Define onde a política será aplicada.

Exemplo: a política de testes será utilizada pelas squads A, B, C e D, estabelecendo diretrizes comuns para as atividades de teste desses times.

3. Objetivos dos testes

Descreve o que a organização pretende alcançar com os testes de software.

Exemplo: reduzir os riscos de que defeitos críticos sejam identificados em produção, por meio da análise antecipada dos requisitos, identificação de riscos e execução de testes adequados.

Os objetivos devem refletir a visão de qualidade adotada pela organização.

4. Processo de testes

Define as atividades relacionadas aos testes e quem é responsável por executá-las.

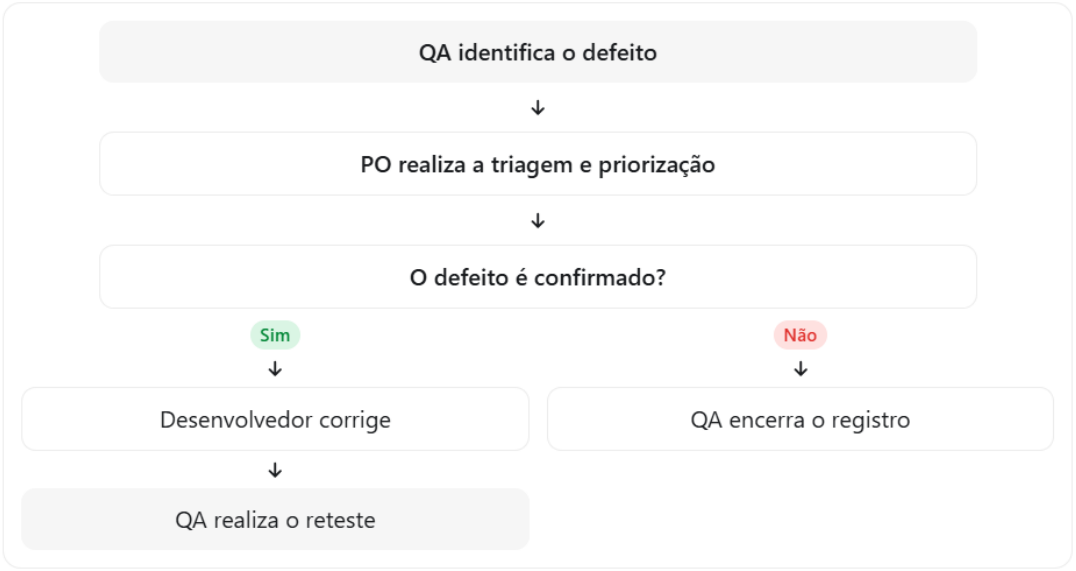
Pode incluir:

- Planejamento e modelagem dos testes.
- Preparação do ambiente.
- Execução dos testes.
- Automação de testes.
- Gestão de defeitos.
- Reteste e regressão.
- Comunicação dos resultados.

O processo pode estar descrito na própria política ou em documentos externos referenciados por ela.

O instrutor sugere o uso do bpmn.io para representar processos por meio de diagramas BPMN, com atividades, responsáveis e pontos de decisão.

Exemplo de fluxo de gestão de defeitos:



Esse é um exemplo simplificado; o fluxo real deve refletir o processo adotado pela empresa.

5. Estrutura da organização de testes

Identifica os papéis e suas responsabilidades em relação à qualidade.

Papel	Responsabilidade relacionada a testes
QA	Planejar, projetar e executar testes, identificar riscos e comunicar resultados.
Líder de QA	Orientar a estratégia, desenvolver profissionais e promover melhorias.
Desenvolvedor	Desenvolver software com qualidade, realizar testes pertinentes e corrigir defeitos.
Product Owner (PO)	Esclarecer necessidades, critérios de aceitação e prioridades do produto.
Scrum Master	Facilitar o trabalho do time e ajudar a remover impedimentos, conforme seu papel no Scrum.

As responsabilidades específicas devem ser adaptadas à realidade da organização.

6. Treinamentos e certificações

Registra as capacitações esperadas para os profissionais envolvidos com testes.

Pode incluir:

- Cursos técnicos.
- Treinamentos internos.
- Certificações profissionais.
- Links para materiais de estudo.
- Competências necessárias para determinadas funções.

Importante: os requisitos de capacitação devem ser definidos de acordo com as necessidades reais da empresa, não necessariamente por meio de uma certificação obrigatória para todos.

7. Mentalidade dos testadores

Apresenta os princípios e comportamentos esperados dos profissionais de QA.

Exemplos:

- Promover a qualidade desde o início do desenvolvimento.
- Ajudar a equipe a identificar problemas antecipadamente.
- Desenvolver conhecimentos técnicos, incluindo leitura e compreensão de código.
- Colaborar com desenvolvedores e demais integrantes do time.
- Pensar criticamente, investigar e identificar riscos.
- Comunicar problemas com clareza e evidências.

O QA não deve atuar apenas como alguém que encontra defeitos no final do desenvolvimento, mas como um profissional que contribui para a qualidade durante todo o ciclo de vida do software.

8. Normas e políticas relacionadas

Relaciona as normas e os documentos que orientam a prática de testes na organização.

Exemplos citados na aula:

- **ISO/IEC/IEEE 29119-3:** documentação de testes de software.
- **ISO/IEC 25010:** modelo de qualidade de sistemas e software.

Também podem ser incluídos links para políticas de desenvolvimento, arquitetura, segurança e outras áreas relevantes.

9. Mensuração do valor dos testes

Define como a organização avalia os benefícios e resultados das atividades de teste.
Uma possibilidade é estimar os custos evitados ao identificar defeitos antes que cheguem à produção.

Exemplo de indicadores:

- Quantidade e criticidade dos defeitos encontrados.
- Defeitos identificados em produção.
- Tempo gasto na correção de defeitos.
- Custos estimados de falhas evitadas.
- Cobertura dos testes, quando aplicável.
- Tempo de execução e manutenção da automação.

Atenção: estimar retorno financeiro exige premissas claras. Nem todo defeito encontrado representa uma economia comprovada, e métricas isoladas não devem ser utilizadas para avaliar toda a qualidade do time.

10. Arquivamento e reutilização de artefatos de testes

Estabelece regras para armazenar, revisar, arquivar e reutilizar os materiais produzidos durante os testes.
Exemplos de artefatos:

- Casos e cenários de teste.
- Scripts automatizados.
- Dados de teste.
- Evidências de execução.
- Relatórios.

A política pode definir critérios de arquivamento e revisão periódica. Por exemplo, casos de teste antigos devem ser avaliados quanto à sua relevância antes de serem mantidos ou arquivados.
A reutilização também pode envolver métodos, funções e componentes automatizados compartilhados entre diferentes testes.

11. Melhoria do processo de testes

Define como a organização identifica problemas e aperfeiçoa continuamente suas práticas de teste.
Um ciclo simplificado de melhoria é:

1. Identificar um problema.
2. Analisar suas causas.
3. Propor uma solução.

4. Avaliar e aprovar a proposta, quando necessário.
5. Implementar a melhoria.
6. Medir os resultados.
7. Compartilhar aprendizados e ajustar o processo.

2. Resumo para revisão

Item	**Pergunta que responde**
Histórico de mudanças	O que foi alterado, por quem e quando?
Escopo	Onde a política se aplica?
Objetivos dos testes	Por que testamos?
Processo de testes	Quais atividades são realizadas e por quem?
Estrutura organizacional	Quais são os papéis e as responsabilidades?
Treinamentos e certificações	Quais competências são esperadas?
Mentalidade dos testadores	Quais princípios orientam o trabalho de QA?
Normas e políticas	Quais referências orientam a organização?
Mensuração de valor	Como avaliamos os resultados dos testes?
Artefatos de testes	Como armazenamos e reutilizamos os materiais?
Melhoria contínua	Como aperfeiçoamos o processo?

3. O que você precisa saber como futura líder de QA
A política de testes não é apenas um documento burocrático. Ela comunica a visão da organização sobre qualidade e estabelece diretrizes para orientar decisões, responsabilidades, processos e melhorias.

Como líder, você precisa saber definir diretrizes claras, alinhar os envolvidos, documentar decisões e garantir que a política seja útil e aplicável à realidade dos times.

A política estabelece orientações gerais; os procedimentos, planos e demais documentos detalham como determinadas atividades serão realizadas.

---

4. ESTRATÉGIA ORGANIZACIONAL

A Estratégia Organizacional de Testes é um documento relacionado à ISO/IEC/IEEE 29119-3 que descreve como os testes devem ser organizados e realizados na organização.

Ela complementa a Política de Testes: enquanto a política estabelece os objetivos e princípios gerais, a estratégia detalha as diretrizes para colocar esses objetivos em prática.

1. Política de testes × Estratégia organizacional de testes

Política de testes	Estratégia organizacional de testes
Define objetivos e princípios gerais.	Define diretrizes para executar os testes.
Explica por que a organização testa.	Explica como os testes são organizados.
Estabelece orientações sobre qualidade.	Detalha níveis, técnicas, ferramentas, métricas e processos.
Aplica-se à abrangência definida pela política.	Orienta as iniciativas de teste dentro de seu escopo.

Exemplo: a política estabelece que a empresa deve reduzir defeitos críticos em produção. A estratégia define como os riscos serão avaliados, quais níveis de teste serão utilizados e como os resultados serão registrados.

2. Estrutura da estratégia organizacional

2.1. Histórico de mudanças

Registra as alterações realizadas no documento, incluindo autor, data, descrição e aprovação, quando aplicável.

2.2. Gerenciamento genérico de riscos

Descreve a abordagem utilizada pela organização para identificar, avaliar, monitorar e tratar riscos.

Pode incluir:

- Identificação dos riscos.
- Avaliação de probabilidade e impacto.
- Priorização dos riscos.
- Planejamento de ações de mitigação.
- Monitoramento das mudanças nos riscos.

O instrutor cita a abordagem PRISMA como exemplo. O importante é documentar a abordagem efetivamente adotada pela organização.

2.3. Seleção e priorização dos testes

Define como a equipe decide o que será testado primeiro e quais técnicas serão utilizadas.

A priorização pode considerar:

- Criticidade da funcionalidade.
- Probabilidade e impacto de falhas.
- Requisitos e critérios de aceitação.
- Histórico de defeitos.
- Complexidade técnica.
- Prazo e recursos disponíveis.

Exemplo: em um sistema bancário, uma transferência financeira pode receber prioridade sobre uma alteração visual de baixo impacto, devido aos riscos envolvidos.

O QA pode trabalhar em conjunto com o Product Owner e os desenvolvedores para compreender as prioridades do produto e definir uma estratégia de testes adequada.

2.4. Documentação e relatórios de testes
Lista os documentos e relatórios que devem ser produzidos ou consultados.

Exemplos:

- Casos e cenários de teste.
- Planos de teste.
- Evidências de execução.
- Relatórios de execução periódicos.
- Relatórios de defeitos.
- Relatórios de cobertura e riscos.

A estratégia indica quais artefatos são esperados e pode incluir links para seus modelos. Ela não precisa conter todos os casos de teste de cada funcionalidade.

2.5. Automação de testes e ferramentas
Define as diretrizes gerais para automação e as ferramentas utilizadas pela organização.

Pode especificar:

- Quais níveis de teste devem ser automatizados.
- Critérios para decidir o que automatizar.
- Padrões de implementação e manutenção.
- Ferramentas e frameworks aprovados.
- Boas práticas de reutilização.
- Integração com pipelines de CI/CD.

A automação deve ser orientada pelo risco, pelo custo de manutenção e pelo benefício esperado, não apenas pela quantidade de testes automatizados.

2.6. Gerenciamento de configuração
Estabelece como os artefatos relacionados aos testes são armazenados, versionados e controlados.

Exemplos:

- Código de testes automatizados armazenado em repositórios Git.
- Documentação mantida em ferramentas corporativas, como o Confluence.
- Controle de versões dos planos e estratégias.
- Identificação das versões de software e dos dados utilizados nos testes.

O objetivo é garantir rastreabilidade, integridade e possibilidade de recuperação dos artefatos.

2.7. Gerenciamento de defeitos
Descreve como os defeitos são registrados, analisados, priorizados, corrigidos e verificados.

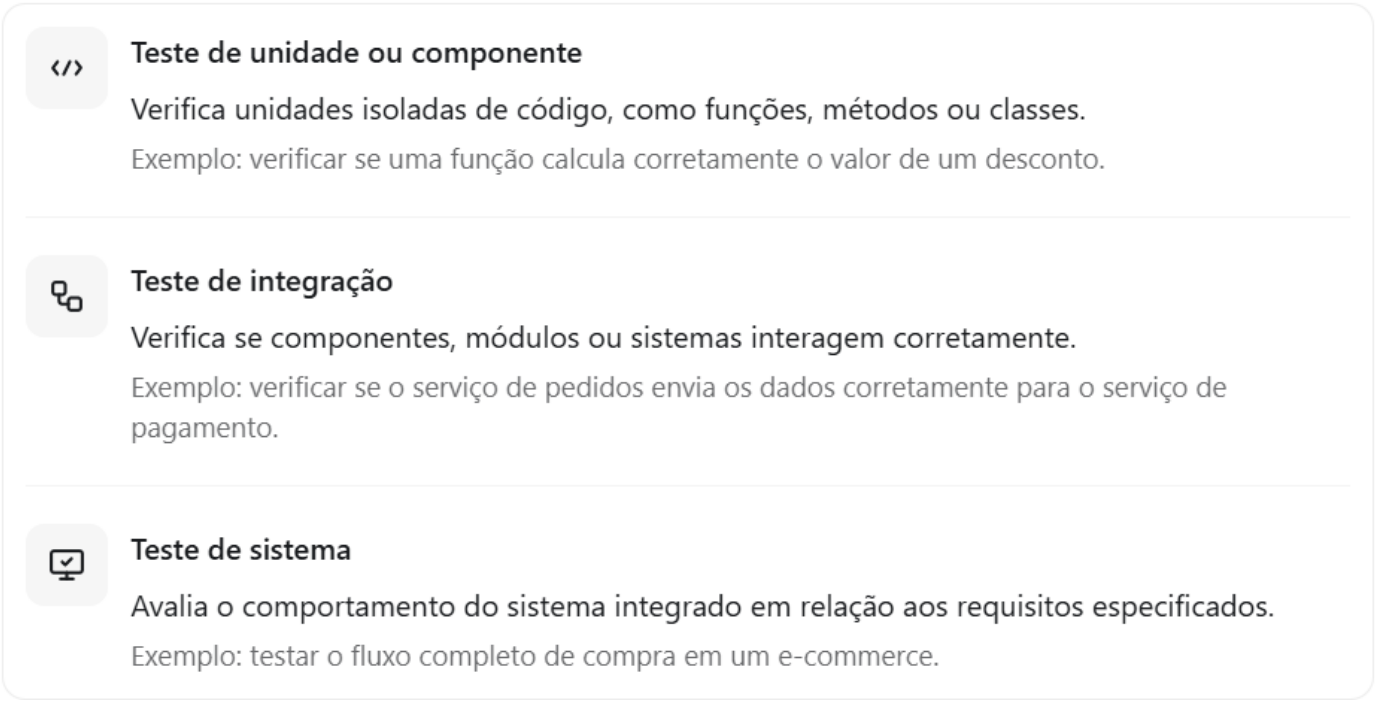
Pode incluir:

- Ferramentas utilizadas para registrar defeitos.
- Informações obrigatórias no registro.
- Critérios de severidade e prioridade.
- Papéis e responsabilidades.
- Fluxo de triagem, correção e reteste.
- Critérios para encerramento.

O processo detalhado pode estar em outro documento, referenciado pela estratégia organizacional.

3. Estratégia por níveis de teste
Um dos pontos centrais da aula é que a estratégia deve detalhar como cada nível de teste funciona na organização.

A ISO/IEC/IEEE 29119-3 trata da documentação de testes. A classificação dos níveis e as técnicas específicas também são abordadas por outras referências, como o ISTQB.



Para cada nível, a estratégia pode documentar os seguintes itens:

Item	**O que deve ser definido**
Processo	Quais atividades são realizadas e em qual sequência.
Critérios de entrada	O que precisa estar pronto para iniciar os testes.
Documentação e relatórios	Quais artefatos e resultados devem ser registrados.
Independência dos testes	Quem testa e qual é sua relação com quem desenvolve.
Técnicas de teste	Quais técnicas são adequadas ao nível.
Ambiente de teste	Onde os testes serão executados e quais dependências existem.
Métricas	Como a execução e a cobertura serão avaliadas.
Reteste e regressão	Como confirmar correções e verificar possíveis efeitos colaterais.

O processo pode ser documentado em tabelas ou diagramas BPMN, desde que fique claro para os envolvidos.

4. Critérios de entrada

Os critérios de entrada são as condições que precisam ser atendidas para iniciar uma atividade de teste.

Exemplos:

- **Unidade**: código implementado e condições mínimas para executar o teste disponíveis.
- **Integração**: componentes integrados, contratos definidos e dependências acessíveis ou simuladas.
- **Sistema**: versão implantada no ambiente apropriado, requisitos e critérios de aceitação disponíveis e ambiente preparado.

Os critérios devem ser definidos de acordo com o contexto; não precisam ser iguais em todos os projetos.

5. Independência dos testes

A independência indica o grau de separação entre quem desenvolve e quem testa.

Em termos gerais:

- O desenvolvedor testa o próprio código: menor independência.
- Outro profissional do mesmo time testa: pode haver maior independência em relação ao autor do código.
- Uma equipe de testes separada testa: maior separação organizacional.
- Uma organização externa realiza testes: pode proporcionar ainda mais independência.

Essa relação não é uma garantia automática de qualidade. Testes realizados pelo próprio desenvolvedor são importantes, e equipes independentes também podem deixar defeitos passar.

A estratégia deve registrar o grau de independência adotado em cada nível, quando pertinente.

6. Técnicas e ambientes de teste

Técnicas de teste
A organização define quais técnicas são apropriadas para cada nível, por exemplo:

- Particionamento de equivalência.
- Análise de valor-limite.
- Tabelas de decisão.
- Testes de transição de estados.
- Cobertura de sentenças e decisões, em testes estruturais.

Ambientes de teste
Também são definidos os ambientes e as dependências necessárias.

Exemplos:

- Testes de unidade executados localmente, com dependências simuladas quando necessário.
- Testes de integração executados em ambiente com os serviços envolvidos.
- Testes de sistema executados em um ambiente controlado, como homologação ou staging.

O uso de dados deve respeitar os requisitos de segurança e privacidade. Dados reais de usuários não devem ser utilizados indiscriminadamente.

7. Métricas, retestes e regressão
As métricas variam conforme o nível de teste.

Nível	**Exemplos de métricas**
Unidade	Cobertura de código, sentenças e decisões.
Integração	Integrações verificadas, falhas de comunicação e contratos validados.
Sistema	Cobertura de requisitos, critérios de aceitação e riscos.

A estratégia também deve explicar como os retestes e os testes de regressão serão realizados:

- **Reteste:** verifica se um defeito específico foi corrigido.
- **Teste de regressão:** verifica se uma mudança provocou problemas em funcionalidades que anteriormente funcionavam.

8. O que você precisa aprender como futura líder de QA
A estratégia organizacional de testes transforma as diretrizes gerais de qualidade em práticas consistentes. Ela não descreve os testes de uma única funcionalidade, mas estabelece como as iniciativas de teste devem ser conduzidas dentro do escopo definido.

Como líder, você precisa saber:

- Definir e comunicar práticas comuns de teste.
- Alinhar a estratégia aos riscos e objetivos do negócio.
- Estabelecer critérios e responsabilidades claros.
- Selecionar métricas úteis para tomada de decisão.

- Promover consistência sem impedir que os times adaptem suas abordagens ao contexto.

Para memorizar: a política define os objetivos e princípios; a estratégia organizacional detalha as diretrizes gerais para alcançá-los; os planos e casos de teste tratam das atividades e verificações específicas de cada iniciativa.

---

5. PLANO E ESTRATÉGIA DE TESTES ADAPTADA

1. Conceito

A ISO/IEC/IEEE 29119-3 aborda a documentação de testes, incluindo o plano de teste e informações relacionadas à estratégia de testes.

O instrutor propõe uma adaptação desses documentos para equipes ágeis, tornando-os mais práticos e menos burocráticos, adequados ao trabalho realizado durante as Sprints.

Diferença entre os documentos estudados:

- **Política de testes:** define os objetivos e princípios gerais da organização.
- **Estratégia organizacional de testes:** estabelece diretrizes gerais para realizar os testes.
- **Plano e estratégia de testes adaptados ao ágil:** organizam o esforço, as condições e as abordagens de teste de uma funcionalidade ou iniciativa específica.

2. Estrutura do plano adaptado ao ágil

2.1. Épico e estimativa inicial

O épico representa uma funcionalidade ou objetivo de maior abrangência.

O documento registra:

- Nome ou identificação do épico.
- Estimativa inicial de complexidade ou esforço.

Essa estimativa é preliminar, feita quando ainda existem poucas informações sobre a funcionalidade.
Exemplo: épico de transferências financeiras — estimativa inicial de complexidade média.

2.2. User Stories e esforço de testes

Depois, são registradas as histórias de usuário (User Stories) que compõem o épico, com uma estimativa mais detalhada do esforço de testes.

Campo	**Descrição**
ID da User Story	Identificador da história.
Descrição	Funcionalidade ou necessidade que será atendida.
Esforço de testes	Estimativa necessária para realizar as atividades de teste.

O esforço pode ser expresso em pontos, horas ou dias, conforme a forma de trabalho adotada pelo time.

Importante: pontos de história são uma medida relativa de esforço e complexidade. Não devem ser tratados automaticamente como horas ou dias.

2.3. Condições de teste e camadas

As condições de teste descrevem, de maneira objetiva, o que precisa ser verificado, sem necessariamente especificar todos os passos de execução.

Exemplo de condição:
Validar transferência de valores para uma conta bancária interna com saldo suficiente.

Essa descrição é mais simples de manter do que um caso de teste extenso com muitos passos, especialmente quando o QA participa desde o início do desenvolvimento e já conhece os requisitos e o comportamento esperado.

Cada condição pode ser registrada com os seguintes campos:

Campo	**Finalidade**
ID	Identificar a condição de teste.
Condição de teste	Descrever o que deve ser verificado.
Resultado esperado	Definir o comportamento correto.
Camada de teste	Indicar em qual nível a condição será verificada.

As camadas podem incluir testes de unidade, integração e sistema, conforme a natureza da condição e a arquitetura da aplicação.

2.4. Exemplo prático
Considere uma User Story de transferência bancária.

ID	**Condição de teste**	**Resultado esperado**	**Camada**
CT-01	Transferir valor com saldo suficiente.	Transferência concluída e saldo atualizado corretamente.	Sistema
CT-02	Tentar transferir valor superior ao saldo disponível.	Operação rejeitada, sem transferência indevida.	Sistema
CT-03	Calcular o valor final da transferência.	Cálculo correto para os valores e regras informados.	Unidade
CT-04	Verificar a comunicação entre o serviço de transferência e o serviço de saldo.	Dados transmitidos corretamente entre os componentes.	Integração

Esses exemplos ilustram como uma condição pode ser associada a uma camada de teste. Na prática, a cobertura pode envolver mais de uma camada quando isso for necessário.

3. Por que adaptar o plano ao Ágil?

Em equipes ágeis, o trabalho é organizado em ciclos curtos, com colaboração contínua entre QA, desenvolvimento e produto.

Por isso, o instrutor propõe:

- Reduzir documentação que não agrega valor ao contexto.
- Evitar cronogramas detalhados de testes que apenas duplicam o planejamento da Sprint.
- Estimar o esforço conforme o conhecimento sobre cada User Story evolui.
- Registrar condições de teste objetivas e fáceis de manter.
- Manter o planejamento alinhado às mudanças nos requisitos.

Isso não significa eliminar planejamento ou documentação. Significa produzir informações suficientes para orientar o trabalho, comunicar riscos e garantir a rastreabilidade necessária.

4. O que você precisa aprender como QA e futura líder

O ponto central desta aula é saber planejar testes sem criar documentação excessiva.

Você precisa conseguir:

1. Entender o épico e suas User Stories.
2. Estimar o esforço de testes de acordo com as informações disponíveis.
3. Definir condições de teste claras.
4. Estabelecer resultados esperados verificáveis.
5. Escolher as camadas de teste adequadas.
6. Ajustar o planejamento conforme o time aprende mais sobre a funcionalidade.

Para memorizar: o plano adaptado ao ágil organiza o que será testado, o esforço necessário, as condições de teste e as camadas envolvidas, mantendo a documentação útil e compatível com a dinâmica das Sprints.

---

6. MELHORIA DO PROCESSO DE TESTES

Para melhorar um processo de testes, é necessário conhecer os fundamentos e as boas práticas de QA. Esse conhecimento permite identificar problemas, propor soluções e justificar os benefícios para a empresa.

Os 7 passos da melhoria do processo

Passo	Etapa	O que fazer
1	Entender o estado atual	Investigar o processo por meio de perguntas a QAs, desenvolvedores, líderes e gestores. Avaliar planejamento, automação, ferramentas, ambientes e participação do QA.
2	Identificar problemas	Analisar as respostas e identificar quais atividades apresentam falhas ou oportunidades de melhoria.
3	Definir ações de melhoria	Estabelecer ações concretas para resolver cada problema identificado.

4	Definir responsáveis	Determinar quem conduzirá cada ação e quem poderá apoiar sua execução.
5	Apresentar à liderança	Organizar os problemas, as ações e os responsáveis em uma tabela para facilitar a análise e a priorização.
6	Implementar as melhorias	Planejar e executar as ações, envolvendo os profissionais necessários e verificando se funcionam.
7	Medir os resultados	Comparar a situação anterior e posterior à mudança, registrar os benefícios observados e apresentá-los à liderança.

Tabela de ações de melhoria do processo

ID	Atividade do Processo Atual	Problema	Ação de Melhoria	Responsável
1	Participação em Reuniões	QA não participa das reuniões desde o início.	Incluir o QA no início do Refinamento.	Scrum Master e o Product Owner
2	Automação de Testes	Testes automatizados ainda não rodam na Pipeline.	Adicionar os testes automatizados mais importantes na Pipeline.	Júlio de Lima
3	Criação de Casos de Teste	QAs investem mais tempo em escrever casos de teste do que de fato testar.	Balancear a criação de casos de teste com a execução de testes exploratórios.	Júlio de Lima

Avaliação dos resultados

Após implementar cada ação, verifique se o problema foi resolvido e quais benefícios foram alcançados.

- Participação do QA: verificar se riscos e ambiguidades são identificados mais cedo.
- Automação: avaliar o tempo de feedback e a ocorrência de defeitos de regressão.
- Testes exploratórios: acompanhar os defeitos relevantes encontrados e o equilíbrio entre documentação e execução.

Conceito-chave: a melhoria do processo de testes segue um ciclo de diagnóstico, identificação de problemas, planejamento de ações, implementação e avaliação de resultados. O objetivo é comprovar que as mudanças trouxeram benefícios reais.

---

7. MENTORIA UM A UM (1:1)

1. Conceito e objetivo

A liderança moderna em QA combina gestão com desenvolvimento individual dos profissionais. O líder atua como mentor, ajudando cada liderado a aprender conceitos, desenvolver habilidades e superar dificuldades específicas.

Mentoria é um acompanhamento individual e personalizado, conduzido por alguém que possui conhecimento no assunto que será desenvolvido.

Exemplos:

- Um QA que precisa aprender automação de testes de API.
- Um profissional que tem dificuldade de comunicação com o time.
- Um testador que precisa melhorar sua capacidade de resolver problemas técnicos.

O mentor deve conhecer o assunto que pretende ensinar. Não é adequado orientar alguém sobre uma competência que ainda não domina.

2. Etapas do processo de mentoria

Etapa	**O que fazer**	**Exemplo prático**
1. Estabelecer o contexto	Identificar a dificuldade, compreender as necessidades do liderado e definir o objetivo da mentoria.	Ajudar um QA a aprender automação de testes de API.
2. Criar um plano personalizado	Organizar os tópicos, as atividades e os objetivos de cada sessão, de acordo com as necessidades da pessoa.	Planejar o estudo de lógica de programação, conceitos de API, testes de API e automação.
3. Conduzir as sessões	Ensinar na prática, fazer perguntas, ouvir as dificuldades e incentivar a participação ativa do mentorado.	Demonstrar como automatizar uma requisição de API e orientar o profissional durante os exercícios.
4. Revisar continuamente o progresso	Definir metas, propor exercícios e avaliar a evolução nas sessões e no trabalho cotidiano.	Solicitar a prática de cinco exercícios e revisar os resultados no encontro seguinte.
5. Finalizar a mentoria	Comparar a evolução com o ponto inicial, avaliar a satisfação do mentorado, reconhecer seu esforço e registrar os resultados.	Verificar se o QA consegue automatizar testes de API com maior autonomia e apresentar os resultados à liderança.

3. Boas práticas do mentor

- Personalização: adaptar o plano à necessidade individual, em vez de aplicar o mesmo treinamento a todos.
- Interação: incentivar perguntas, participação e resolução prática de problemas.
- Metas claras: definir atividades e resultados esperados para cada sessão.
- Acompanhamento: observar a evolução, identificar dificuldades e ajustar o plano quando necessário.
- Confidencialidade: preservar as dificuldades pessoais compartilhadas pelo mentorado.
- Compartilhamento de conhecimento: quando houver aprendizados úteis para todo o time, compartilhar o conteúdo sem expor a identidade ou as dificuldades individuais do profissional.

4. Exemplo de plano de mentoria em QA

Sessão	Conteúdo	Atividade prática	Resultado esperado
1	Lógica de programação	Resolver exercícios de lógica.	Compreender os fundamentos necessários para programar.
2	Conceitos de API	Analisar requisições e respostas.	Entender como uma API funciona.
3	Testes de API	Elaborar e executar testes manuais.	Validar respostas, dados e comportamentos da API.
4	Automação de API	Automatizar testes com uma ferramenta adequada.	Criar testes automatizados básicos.
5	Revisão e avaliação	Executar um desafio prático e revisar os resultados.	Demonstrar evolução e identificar próximos passos.

Esse é um exemplo de estrutura: a quantidade de sessões e os conteúdos devem ser ajustados ao conhecimento prévio do profissional.

Conceito-chave: na liderança contemporânea, desenvolver pessoas é uma responsabilidade importante. A mentoria transforma uma dificuldade individual em um plano estruturado de aprendizagem, com prática, acompanhamento e avaliação dos resultados.

---

8. ESTIMATIVA DE ESFORÇO EM TESTES

1. Conceito
A estimativa de três pontos calcula o esforço de testes de uma User Story considerando três cenários: otimista, mais provável e pessimista. A técnica ajuda a considerar incertezas, como defeitos, indisponibilidade de ambientes e falta de dados de teste.

2. Cenários

Cenário	Descrição
Otimista (O)	Tudo ocorre conforme o planejado.
Mais provável (M)	Condições habituais do time, incluindo problemas comuns.
Pessimista (P)	Dificuldades significativas, atrasos e impedimentos.

3. Fórmulas

Estimativa PERT:

$$E = \frac{O + 4M + P}{6}$$

Desvio padrão aproximado:

$$\sigma = \frac{P - O}{6}$$

4. Exemplos

User Story	Otimista	Mais provável	Pessimista	Estimativa	Desvio padrão
JIRA-6776	5 pontos	8 pontos	13 pontos	8,33	1,33
JIRA-8998	2 h	2,5 h	8 h	3,33 h	1 h

5. Como calcular no Excel

Monte uma planilha com estas colunas:

Coluna	Cabeçalho
A	ID
B	User Story
C	Otimista (O)
D	Mais provável (M)
E	Pessimista (P)
F	Estimativa PERT
G	Desvio padrão

Preencha as linhas 2 e 3 com os exemplos abaixo:

ID	User Story	Otimista	Mais provável	Pessimista
1	JIRA-6776	5	8	13
2	JIRA-8998	2	2,5	8

Na célula F2, digite a fórmula da estimativa:

=(C2+4*D2+E2)/6
Na célula G2, digite a fórmula do desvio padrão:
=(E2-C2)/6

Depois, arraste as fórmulas para as linhas seguintes para calcular as demais User Stories automaticamente.

Resultados esperados:

- JIRA-6776: estimativa de 8,33 pontos e desvio padrão de 1,33 ponto.
- JIRA-8998: estimativa de 3,33 horas e desvio padrão de 1 hora.

Dica: formate as células F e G com duas casas decimais. Não misture horas e pontos na mesma escala de estimativa; mantenha a unidade de medida consistente em cada conjunto de cálculos.

6. Aplicação em QA

A técnica ajuda a estimar o esforço de testes, planejar sprints e comunicar incertezas à gestão. O resultado é uma estimativa fundamentada em cenários, não uma garantia do tempo real necessário.

---

9. ESTRUTURANDO O FEEDBACK PARA SEUS LIDERADOS

1. Conceito
O framework SBI ajuda líderes a fornecer feedback de forma objetiva, estruturada e baseada em fatos, principalmente quando precisam abordar comportamentos que devem ser melhorados.

SBI significa:

- **S — Situation (Situação):** quando e onde aconteceu?
- **B — Behavior (Comportamento):** o que a pessoa fez ou disse, de forma observável?
- **I — Impact (Impacto):** quais foram as consequências desse comportamento?

Após o SBI, pode-se definir o Path Forward (caminho adiante), estabelecendo ações para melhorar o comportamento no futuro.

2. Etapas do feedback

Etapa	O que fazer	Exemplo
1. Situation — Situação	Descrever o contexto com data, reunião, participantes e assunto.	Na reunião com o cliente, em 4 de julho de 2025, discutimos problemas de produção.
2. Behavior — Comportamento	Descrever o que a pessoa fez, sem julgamentos ou rótulos pessoais.	Você elevou a voz e criticou os desenvolvedores diante do cliente.
3. Impact — Impacto	Explicar as consequências observadas ou verificáveis.	O cliente demonstrou preocupação com a integração do time e encerrou a reunião insatisfeito.

4. Path Forward — Caminho adiante	Combinar como agir em situações semelhantes e definir ações de melhoria.	Discutir divergências com o time em particular e alinhar com a liderança como tratar problemas recorrentes de produção.

3. Como preparar um feedback

Antes da conversa, o líder deve:

1. Reunir fatos e dados que sustentem o feedback.
2. Identificar a situação específica e o comportamento observado.
3. Verificar o impacto real, evitando atribuir consequências sem evidências.
4. Organizar a mensagem usando o SBI.
5. Conversar em particular, ouvir a perspectiva do profissional e definir próximos passos em conjunto.

Importante: feedback não deve ser usado para constranger a pessoa ou impedir que ela discorde. O objetivo é tornar o comportamento e suas consequências compreensíveis e favorecer a melhoria.

4. Registro de feedbacks

Quando necessário, o líder pode utilizar uma planilha para acompanhar feedbacks e ações combinadas.

ID	Situação	Comportamento	Impacto	Plano de ação
1	Reunião com cliente sobre falhas em produção	QA elevou a voz e criticou desenvolvedores diante do cliente	Cliente demonstrou preocupação com a colaboração do time	Tratar divergências internamente e estabelecer um canal para discutir falhas recorrentes

Os registros devem ser objetivos, confidenciais e limitados às informações necessárias.

Conceito-chave: o SBI organiza o feedback em situação, comportamento e impacto. O Path Forward complementa o processo, transformando a conversa em um plano de melhoria para o futuro.

---

10. CRIANDO UM BUSINESS CASE

1. O que é um Business Case?

Um Business Case é um documento que apresenta e justifica uma proposta de projeto ou mudança, demonstrando o problema, a solução sugerida, os custos, os benefícios e os motivos pelos quais a iniciativa deve ser aprovada.

Na área de QA (Quality Assurance), ele ajuda a demonstrar como um investimento em qualidade pode contribuir para os objetivos do negócio.

Objetivo principal: convencer os responsáveis pela decisão de que a proposta é necessária, viável e vantajosa.

2. As 6 etapas de um Business Case

1. Sumário executivo
Apresente uma visão geral da proposta, o problema, a solução e o resultado esperado. Deve ser objetivo e facilitar a compreensão rápida do projeto.

2. Propósito, problema e solução
Explique por que a iniciativa é necessária, qual problema será resolvido e qual solução está sendo proposta.

3. Opções disponíveis
Compare as alternativas possíveis para resolver o problema, considerando suas vantagens e desvantagens.

4. Curso de ação, custo-benefício e prioridade
Justifique a alternativa escolhida, avaliando custos, benefícios, urgência e prioridade para o negócio.

5. Estratégia de implementação
Defina como a proposta será colocada em prática, incluindo as ações necessárias e a organização da execução.

6. Aprovação dos stakeholders
Apresente a proposta às partes interessadas e aos responsáveis pela decisão, buscando aprovação e alinhamento.

3. Aplicação prática em QA

A aula utiliza como exemplo a necessidade de contratar profissionais de QA adicionais para atender a três squads.

O Business Case deve demonstrar:

- Problema: a necessidade de ampliar a capacidade de QA.
- Solução: contratar profissionais para atender à demanda.
- Alternativas: contratação direta ou contratação por consultoria.
- Análise: comparar custos, benefícios e prioridades de cada opção.
- Implementação: organizar as etapas necessárias para executar a alternativa escolhida.
- Aprovação: apresentar a justificativa aos responsáveis pela decisão.
-

O importante não é apenas pedir mais profissionais, mas demonstrar por que a contratação é necessária e qual alternativa atende melhor às necessidades do negócio.

4. Principais conceitos para memorizar

Conceito	**O que lembrar**
Business Case	Justifica uma proposta com argumentos de negócio.
Problema	Explica a necessidade que motivou a iniciativa.

Solução	Apresenta a proposta para resolver o problema.
Alternativas	Mostram as opções possíveis.
Custo-benefício	Ajuda a avaliar se os benefícios justificam os custos.
Curso de ação	Define a alternativa escolhida e justifica a decisão.
Estratégia de implementação	Define como a iniciativa será executada.
Stakeholders	São as partes interessadas que participam ou influenciam a decisão.

Atenção: um Business Case não deve apenas descrever uma solução. Ele precisa justificar a escolha e demonstrar seu valor para o negócio.

---

11. MÉTRICAS EM TESTE DE SOFTWARE

1. TAXA DE DEFEITOS ENCONTRADOS EM PRODUÇÃO

O que mede: a proporção de defeitos identificados em produção em relação ao total de defeitos registrados no período.

Fórmula:
Taxa (%) = (Defeitos encontrados em produção ÷ Total de defeitos) × 100

Exemplo: 4 defeitos em produção entre 10 defeitos registrados.
Cálculo: (4 ÷ 10) × 100 = **40%**

Objetivo: reduzir os defeitos que chegam ao usuário e investigar como poderiam ter sido detectados antes.

2. TAXA DE RETESTE

O que mede: a proporção de execuções adicionais de testes. As repetições podem acontecer devido a defeitos ou problemas no ambiente.

Fórmula apresentada na aula:
Taxa (%) = (Execuções adicionais ÷ Total de execuções) × 100
Execuções adicionais por caso = Quantidade de execuções − 1

Exemplo: 8 execuções adicionais em 18 execuções no total.
Cálculo: (8 ÷ 18) × 100 = **44,4%**

Atenção: investigue os motivos das reexecuções e documente a fórmula adotada pela equipe.

3. CUSTO DOS DEFEITOS

O que mede: estima o custo do trabalho necessário para tratar um defeito durante as etapas do processo.

Fórmulas:
Custo por papel = Horas trabalhadas × Custo por hora
Custo total = Soma dos custos de todos os papéis envolvidos

Como calcular:

1. Registrar as horas gastas por QA, desenvolvimento, DevOps e outros profissionais envolvidos.
2. Multiplicar as horas pelo respectivo custo por hora.
3. Somar os valores para obter o custo total.

Atenção: a transcrição original mistura referências a reais e dólares, e o total informado não coincide claramente com os dados. Defina uma única moeda e valide os valores antes de utilizar o exemplo.

4. COBERTURA DE AUTOMAÇÃO DE TESTES

O que mede: quanto do conjunto de testes definido para automação já foi automatizado.

Fórmula:
Cobertura (%) = (Casos automatizados ÷ Total de casos no escopo de automação) × 100

Exemplo: 4 casos automatizados de um total de 6.
Cálculo: (4 ÷ 6) × 100 = **66,7%**

Atenção: considere apenas os casos que fazem parte do escopo definido para automação, por exemplo, aqueles marcados como “deve automatizar = sim”.

5. COBERTURA DE TESTES

O que mede: a proporção dos casos de teste planejados que já foram executados. Pode ser analisada por user story, requisito, risco ou outro artefato.

Fórmula por artefato:
Cobertura (%) = (Casos executados ÷ Casos planejados) × 100

Fórmula geral:
Cobertura geral (%) = (Total de casos executados ÷ Total de casos planejados) × 100

Exemplo: 11 casos executados de um total de 15 planejados.
Cálculo: (11 ÷ 15) × 100 = **73,3%**

Importante: a cobertura mostra quanto do planejamento foi executado, mas não comprova, sozinha, que o sistema está livre de defeitos.

6. QUALIDADE DA RELEASE

O que mede: a aula apresenta um indicador composto por duas variáveis: casos de teste falhos e defeitos encontrados. Os pesos e limites aceitáveis devem ser definidos pela equipe.

Valores do exemplo da aula:

- Peso dos casos falhos: 0,60.
- Limite aceitável de casos falhos: 5.
- Peso dos defeitos encontrados: 0,40.
- Limite aceitável de defeitos: 3.

Fórmula apresentada:
Índice = (1 − Casos falhos ÷ Limite de casos falhos) × 0,60

- (1 − Defeitos encontrados ÷ Limite de defeitos) × 0,40

Atenção: os resultados numéricos falados na transcrição não coincidem com a fórmula e os dados apresentados. A fórmula também pode produzir resultados negativos quando os limites são ultrapassados. Antes de utilizar esse indicador, a equipe deve validar o cálculo e definir como interpretar o resultado.

7. TEMPO MÉDIO PARA DETECÇÃO E CORREÇÃO DE DEFEITOS
O que mede: o tempo necessário para resolver os defeitos analisados.

Fórmulas:
Tempo de resolução = Data/hora de fechamento − Data/hora de abertura
Tempo médio = Soma dos tempos de resolução ÷ Quantidade de defeitos

Como analisar:

- Definir o período da análise, por exemplo, os últimos 30 dias.
- Verificar os defeitos que demoraram mais para serem resolvidos.
- Se necessário, filtrar por severidade ou prioridade.
- Investigar as causas dos atrasos.

Atenção: as datas da transcrição estão incompletas, portanto não é possível confirmar a média numérica do exemplo. O cálculo demonstrado mede o intervalo entre a abertura e o fechamento do defeito. Para medir o tempo de detecção separadamente, são necessários outros dados.

CONSIDERAÇÕES FINAIS
As métricas ajudam a acompanhar a qualidade do software e identificar oportunidades de melhoria.
Para utilizá-las corretamente:

- Defina claramente cada métrica e sua fórmula.
- Mantenha os dados consistentes e rastreáveis.
- Estabeleça o período de análise.
- Interprete os resultados considerando o contexto e os riscos.
- Investigue as causas dos resultados, em vez de observar apenas os percentuais.
- Revise os indicadores com a equipe para verificar se continuam relevantes.

As sete métricas estudadas:

1. Taxa de defeitos encontrados em produção.
2. Taxa de reteste.
3. Custo dos defeitos.
4. Cobertura de automação de testes.
5. Cobertura de testes.
6. Qualidade da release.
7. Tempo médio para detecção e correção de defeitos.

---

12. ESTRUTURANDO POCS (PROVAS DE CONCEITO)

Exemplo: Supertest para testes de API REST

1. Conceito de POC

Uma **Prova de Conceito (POC)** é um pequeno projeto realizado para avaliar se uma ferramenta, um processo ou uma metodologia atende às necessidades de uma empresa.

Aspecto	**Descrição**
Objetivo	Verificar se uma solução é adequada ao contexto da empresa.
Planejamento	Definir metas alcançáveis e mensuráveis.
Organização	Determinar atividades, responsáveis e critérios de sucesso.
Resultado	Produzir evidências que apoiem a decisão de implementar ou não a solução.

Uma POC desorganizada pode consumir tempo sem apresentar resultados concretos. Por isso, deve ser planejada e documentada.

2. As cinco etapas da POC

Etapa	**Descrição**
1. Definição dos objetivos	Estabelecer o que se deseja alcançar e como medir o sucesso.
2. Definição do escopo	Listar as atividades, os responsáveis e os critérios de sucesso.
3. Execução	Realizar as atividades previstas no escopo.
4. Análise dos resultados	Analisar os dados obtidos e registrar o feedback.
5. Documentação dos resultados	Consolidar as conclusões e apresentá-las à gestão.

3. Exemplo prático: Supertest × ReadyAPI

A aula apresenta uma POC hipotética para avaliar se a biblioteca open source **Supertest** pode substituir o ReadyAPI nos testes de API REST.

3.1. Objetivos

Objetivo	Meta apresentada
Substituir a ferramenta atual	Avaliar se o Supertest atende aos testes de API REST realizados no ReadyAPI.
Reduzir custos	Eliminar os custos de licenciamento da ferramenta atual.
Ampliar a automação	Aumentar de 5 para 20 o número de profissionais que utilizam a ferramenta.

A aula menciona US$ 1.500 por ano e, posteriormente, calcula US$ 7.500 por ano considerando cinco licenças ou usuários. A transcrição não esclarece completamente essa diferença.

3.2. Escopo

O escopo define o que será realizado durante a POC. Cada atividade deve apresentar quatro informações:

Campo	Finalidade
ID	Identificar a atividade.
Descrição	Informar o que será realizado.
Responsáveis	Indicar quem executará a atividade.
Critério de sucesso	Definir como verificar se a atividade foi concluída corretamente.

Atividades do exemplo:

ID	Atividade	Responsável	Critério de sucesso
1	Identificar os scripts mais complexos da suíte de testes de API REST no ReadyAPI.	José	Scripts selecionados conforme a avaliação do time de QA: Maria, José e Júlio.
2	Verificar se os scripts existentes no ReadyAPI podem ser reescritos no Supertest.	José e Maria	Os scripts considerados mais complexos são reescritos e executados com sucesso.
3	Outras atividades da POC.	A definir	A definir conforme o planejamento.

O terceiro item está incompleto no documento original.

3.3. Execução

Item	Informação
Início previsto	5 de julho de 2025.
Análise e documentação para discussão	15 de julho de 2025.
Participantes	Maria, José e Júlio.

Dedicação	Duas horas por dia para cada participante durante o período da POC.

Nessa etapa, a equipe executa as atividades planejadas e registra as evidências obtidas.

3.4. Análise dos resultados

No exemplo, os scripts são analisados conforme sua complexidade.

Critério analisado	**Exemplos citados na aula**
Funções	Funções utilizadas nos scripts.
Métodos	Variação dos métodos utilizados.
Recursos externos	Conexão com banco de dados.
Relatórios	Geração de relatórios de testes.
Integrações	Integração com ferramentas de gestão de testes.

A aula prevê registrar a lista dos scripts selecionados e, posteriormente, documentar os resultados de sua reescrita no Supertest.

O texto referente ao item 2 da análise está em aberto no documento original, portanto seus resultados detalhados não foram informados.

3.5. Documentação dos resultados

A documentação final reúne as conclusões da POC para discussão com a gestão.

Resultado apresentado no exemplo	**Conclusão**
Substituição do ReadyAPI	O Supertest pode ser utilizado para substituição.
Recursos complementares	Serão necessárias outras bibliotecas para conexão com banco de dados e integração com ferramentas de gestão de testes.
Licenciamento	O custo apresentado passa de US$ 7.500 para US$ 0 por ano.
Ampliação da automação	O número de profissionais poderá passar de 5 para 20.
Aceitação da ferramenta	Os desenvolvedores demonstraram conforto com o Supertest por já programarem em JavaScript.

Esses resultados fazem parte do cenário hipotético da aula, não de uma validação real da ferramenta.

4. Diferença entre escopo, análise e documentação

Seção	**Pergunta que responde**
Escopo	O que será feito, por quem e como o sucesso será avaliado?

Análise dos resultados	O que aconteceu durante a execução e quais evidências foram obtidas?
Documentação dos resultados	Quais foram as conclusões finais e quais decisões podem ser tomadas?

5. Principais aprendizados

- Uma POC deve ter objetivos claros e mensuráveis.
- O escopo precisa identificar as atividades, os responsáveis e os critérios de sucesso.
- A execução deve seguir o planejamento.
- Os dados devem ser analisados durante a POC.
- Os resultados precisam ser documentados e apresentados à gestão.
- A conclusão da POC serve de base para discutir a possível implementação da solução.

Conclusão

A POC permite avaliar uma ferramenta, um processo ou uma metodologia de forma organizada antes de decidir por sua implementação.

As cinco etapas são: **definir objetivos, estabelecer o escopo, executar, analisar os resultados e documentar as conclusões**.

No exemplo da aula, essas etapas são utilizadas para avaliar a substituição do ReadyAPI pelo Supertest em testes de API REST, considerando custos, funcionalidades e ampliação da automação.

---

13. SÍNTESE DO APRENDIZADO

Neste módulo, aprendi que a liderança em testes de software envolve muito mais do que acompanhar a execução dos testes. O líder de QA precisa desenvolver pessoas, orientar decisões, melhorar processos e alinhar as práticas de qualidade aos objetivos da organização. A liderança contemporânea valoriza a autonomia, a colaboração e o desenvolvimento profissional, ajudando os QAs a se tornarem capazes de identificar riscos, resolver problemas e tomar decisões com mais independência.

Compreendi a importância da **Política de Testes**, que estabelece os objetivos, princípios e diretrizes gerais de qualidade, e da **Estratégia Organizacional de Testes**, que define como os testes devem ser organizados, considerando riscos, técnicas, ferramentas, ambientes, métricas e responsabilidades. Também aprendi que o planejamento de testes adaptado ao Ágil deve ser objetivo e flexível, organizando o esforço, as condições de teste e as camadas de verificação sem criar documentação desnecessária.

Outro aprendizado importante foi o processo de melhoria contínua dos testes, que começa pela análise do cenário atual, passa pela identificação de problemas e definição de ações e responsáveis, e termina com a implementação e a avaliação dos resultados. Nesse contexto, a mentoria individual permite desenvolver competências de forma personalizada, enquanto o framework SBI, complementado pelo Path Forward, ajuda a fornecer feedbacks baseados em fatos e a estabelecer ações concretas de melhoria.

Também estudei técnicas e ferramentas para apoiar o planejamento e a tomada de decisões. A estimativa de três pontos considera cenários otimista, mais provável e pessimista para estimar o esforço de testes e

suas incertezas. O Business Case, por sua vez, permite justificar propostas, comparar alternativas, avaliar custos e benefícios e demonstrar o valor de uma iniciativa para o negócio.

Aprendi ainda que as métricas de testes ajudam a acompanhar resultados e identificar oportunidades de melhoria. Entre as métricas estudadas estão a taxa de defeitos encontrados em produção, a taxa de reteste, o custo dos defeitos, a cobertura de automação, a cobertura de testes, a qualidade da release e o tempo médio para detecção e correção de defeitos. Para utilizá-las corretamente, é necessário definir fórmulas claras, manter dados consistentes e interpretar os resultados considerando o contexto, os riscos e os objetivos da equipe.

Por fim, compreendi que a Prova de Conceito (POC) é utilizada para avaliar a viabilidade de uma solução antes de decidir por sua implementação. Ela deve seguir cinco etapas: definição dos objetivos, estabelecimento do escopo, execução, análise dos resultados e documentação das conclusões. O exemplo da avaliação do Supertest como possível substituto do ReadyAPI demonstrou como uma POC pode apoiar decisões relacionadas a funcionalidades, custos, ferramentas complementares e ampliação da automação.

Concluo que atuar como uma profissional de QA preparada para assumir responsabilidades de liderança exige combinar conhecimento técnico, planejamento, comunicação, análise de resultados e desenvolvimento de pessoas. O objetivo é contribuir para a melhoria contínua dos processos, apoiar decisões baseadas em evidências e demonstrar o valor da qualidade para a organização.
